

Inigo Quilez :: articles :: hacking a generic ray-intersector - 2018



Intro

It's 2018, and I'm thinking about the shiny hardware accelerated ray-triangle intersector embedded in NVidia's new GPUs, and I'm asking myself the following – can we repurpose an ray-triangle intersector for anything else than ray-triangle intersections? Seems like intersecting a ray and triangle is a pretty specialized task, so at first glance one might think no. But it would be nice if it could be reused for something else. Of course, I have no clue about how NVidia implemented theirs or what their SDK looks like (yet), but I thought I'd tackle it from a mathematical perspective at least, and actually hack a generic software ray-triangle intersector into doing something different.

Part 1 - intersection

So, triangles are cool. But they are the easy part of rendering a film shot. It's hair, fur, grass and cloth that take all the cycles. These are all curve primitives, not triangles. Cubic Bezier (or Catmull-Rom) segments and linear segments are usually employed in film. When it comes to ray-tracing them, there are many options. On the fly tessellation into triangles for regular ray-triangle intersection test is one option. But maybe not the best – for example one can compute the closest distance between the ray and curve segments. This allows for good antialiasing, since the renderer can expand the curve thickness dynamically to match a pixel's footprint so the primitive gets always rasterized/raysampled. Of course, the curve's artificially inflated contribution to the pixel has to be adjusted by compensating its coverage by opacity. This inflation can be done with normal triangle tessellation too (Renderman has been doing this since 1997 in the context of rasterization and it works great assuming you have a powerful Order Independent Transparency in place such as per pixel/fragment linked lists or Alpha to Coverage). But computing the closest distance between ray and curve can save lots of tessellation (which means savings in computation and storage).

One more reason to be able to compute closest distances from a ray to a linear segment are neon lights, negative lights/occlusion tweaks, lasers and many other effects. Although one can argue that such computation enable “tricks” that are not physically based, I think having the option to break outside the limits of

a intersectors and putting others mechanisms for querying global scene information to developers can only bring good things to the field, just like opening programmable shading helped escape the dead-end the hardware Transform and Lighting hardware created. Not that I think hardcoded BVH+ray_tri_intersect() are a dead-end just yet, we still have 4 or 5 years to learn to use it well.

Anyways, now, let's see if we can repurpose a generic ray-triangle intersector to compute ray-segment distances, if only for seeing how far such an exercise brings us.

Let's note that a ray-triangle intersector is solving following equation: $p(t) = r_o + t \cdot r_d$ for the ray and $q(u,v) = v_0 + u \cdot (v_1 - v_0) + v \cdot (v_2 - v_0)$ for the triangle simultaneously. So we can just equate them:

$$p(t) = q(u,v)$$

which leads to:

$$r_o + t \cdot r_d = v_0 + u \cdot (v_1 - v_0) + v \cdot (v_2 - v_0)$$

where t being the distance along the ray starting at position r_o and traveling in (not necessarily normalized) direction r_d , where u and v being the barycentric coordinates of the intersection point in the triangle, and v_0 , v_1 and v_2 are naturally the three vertices of the triangle.

Even though this looks like a single equation, because we are rendering in 3D and hence the three vertices have three coordinates, this really represents a system of three linear equations (in x , y and z) with three unknowns (u , v and t). As such, solving the intersection involves four 3×3 determinants and one division (by Cramer's rule). Something like this:

```
vec3 ray_tri_intersect( in vec3 ro, in vec3 rd, in vec3 v0, in vec3 v1, in vec3 v2,
{
    vec3 v1v0 = v1-v0, v2v0 = v2-v0, rov0 = ro-v0;

    float d = 1.0/determinant(mat3(v1v0, v2v0, -rd));
    float u = d*determinant(mat3(rov0, v2v0, -rd));
    float v = d*determinant(mat3(v1v0, rov0, -rd));
    float t = d*determinant(mat3(v1v0, v2v0, rov0));

    return (u < 0.0 || u > 1.0 || v < 0.0 || (u+v) > 1.0) ? vec3(-1) : vec3( u, v, t);
}
```

By noticing that determinants are really volumes that can therefore be computed as a dot and cross product of

the basis vectors in the matrix, and noticing that many of the vectors in the four 3x3 matrices are repeated, one can optimize that to

```
vec3 ray_tri_intersect( in vec3 ro, in vec3 rd, in vec3 v0, in vec3 v1, in vec3 v2
{
    vec3 v1v0 = v1-v0, v2v0 = v2-v0, rov0 = ro-v0;

    vec3 n = cross( v1v0, v2v0 );
    vec3 q = cross( rov0, rd );
    float d = 1.0/dot( n, rd );
    float u = d*dot( -q, v2v0 );
    float v = d*dot( q, v1v0 );
    float t = d*dot( -n, rov0 );

    return (u<0.0 || v<0.0 || (u+v)>1.0) ? vec3(-1) : vec3( u, v, t );
}
```

although that is really not important to the purpose of this article, just wanted to drop the optimization in quickly before I get messages from people letting me know it can be optimized. The important part here is that a ray-triangle intersector is really a 3x3 linear system solver, if we ignore the last test for the range of the barycentric coordinates (the conditional move at the end of the function).

By the way, quick digression - that condition could be changed to $u^2+v^2 \leq 1$ to have a ray-disk intersector, which could be great to have in hardware as well for point clouds based occlusion or reflections (I really hope some NVidia guys are working on that tiny extension).

In any case, it seems that most of the code in ray_tri_intersect(), or circuitry in the case of a hardware implementation, should in theory be reusable for anything that needs a 3x3 linear solver, shouldn't it?

Part 2 - distance

Now back to our line-segment distance problem for hair/fur/grass/cloth curve rendering. We want to solve the closest distance between a ray and a linear segment. This can be expressed as the minimization of

$$|p(t) - q(s)|$$

where $p(t)$ is again a point in the ray of the form $p(t) = ro + t*rd$, and where q is in the segment and takes the

form $q(s) = v_0 + (v_1 - v_0)s$.

We can assume $|rd|=1$, $t>0$, $0 \leq s \leq 1$. By taking actual distances squared and applying partial derivatives on s and t and equating those to zero (to find the function's minimum), we get a system of two linear equations with two variables (s and t). That can be solved with Cramer's rule again, and optimized down to the following:

```
vec3 ray_seg_distance( in vec3 ro, in vec3 rd, in vec3 v0, in vec3 v1 )
{
    vec3 ba = v1-v0, oa = ro-v0;

    float a = dot(ba,ba);
    float b = dot(rd,ba);
    float c = dot(oa,ba);
    float e = dot(oa,rd);

    vec2 st = vec2( c-b*e, b*c-a*e)/(a-b*b);

    st.x = min( max( st.x, 0.0), 1.0 );
    st.y =      max( st.y, 0.0 );

    return vec3( distance( v0 + st.x*ba,
                          ro + st.y*rd ), st );
}
```

With the caveat of course that usually one doesn't need to compute the square root in distance() unless the proximity of the ray to the segment is indeed smaller than the pixel footprint or ray differential.

In any case, we need to see if this 2x2 system of linear equations can be converted into the type of 3x3 system of equations that a ray intersector solves. And it turns out that we got really lucky and indeed it can be done. All we need to do is transpose a few things (which doesn't change the value of the determinants) and add some padding to fit the 2x2 data into the 3x3 layout the intersector expects. It looks like this:

```
vec3 ray_seg_distance( in vec3 ro, in vec3 rd, in vec3 v0, in vec3 v1 )
{
    mat3x3 ma = mat3( v1-v0, -rd, vec3(0) );

    vec3 k1 = (ro-v0)*ma;
    vec3 k2 = (v1-v0)*ma;

    vec2 st = ray_tri_intersect( k1, vec3(0,0,-1), vec3(0,0,0), k2, vec3(k2.y,1,0) );

    st.x = min( max( st.x, 0.0), 1.0 );
    st.y =      max( st.y, 0.0 );
}
```

```
return vec3( distance( v0 + st.x*(v1-v0),  
                      ro + st.y*rd ), st );  
}
```

Conclusion

Note that much of the data passed to the triangle intersector are zeros and ones, which is a waste of computations and an un-optimization when done over a software implementation of ray_tri_intersect. However, if ray_tri_intersect is implemented in hardware there are three different ways I can see things could go from here:

[1] ray_tri_intersect() is in hardware, ray_seg_distance() is kept in software. Not clear to me the savings in the dot products, the multiplications and the division are worth the latency of having to wait for ray_tri_intersect() to compute and return its results to the shader. Maybe it is and this is a win already!?

[2] ray_tri_intersect() is in hardware, and ray_seg_distance() is made in hardware too as some extra circuitry around ray_tri_intersect(). While all that padding that we need to augment the 2x2 system into a 3x3 is wasted computations in the form of multiplications by zero and one, it does however reuse some silicon and save dice space.

[3] ray_seg_distance() is slow even in hardware, we need to remove all those useless multiplications by zero and one. We need custom circuitry, and we'll just pay the price in the dice space. Good try.

Even in case 3, I think the point of this exercise was to see if a general ray intersector can be hacked into helping computations different from just intersecting a ray with a triangle, from a mathematical perspective. And the answer is "seems like yes!"

inigo quilez - learning computer graphics since 1994